

Autenticación de Usuarios con Base de Datos Relacional

Solsticio: registro, inicio de sesión con bcrypt sobre MySQL, tokens JWT y control de acceso por roles

Presentado por:

Joan Sebastián Lara Fuenmayor
Miguel Ángel Marín Baracaldo

Docente:

Leandro Pájaro Fuentes

Fundación Universitaria Konrad Lorenz

Facultad de Matemáticas e Ingenierías

Ingeniería de Sistemas

Nuevas Tecnologías de Desarrollo

Bogotá D.C., 5 de noviembre de 2026

Índice

1. Introducción **3**

2. Objetivos **3**

3. Modelo relacional de la autenticación **3**

4. Script modificado: qué cambió en el código **4**

4.1 Registro contra MySQL 4

4.2 Inicio de sesión y hashing 5

4.3 Token JWT y protección de rutas 6

5. Roles y permisos (RBAC) **6**

6. Instrucciones para ejecutar **8**

6.1 Descargar el proyecto 8

6.2 Crear la base de datos 8

6.3 Entorno virtual y dependencias 8

6.4 Variables de entorno 8

6.5 Primer administrador y datos de ejemplo 8

6.6 Levantar el servidor 8

6.7 Ejecutar el script de prueba y las pruebas automáticas 8

7. Evidencias de la ejecución **10**

7.1 Script de prueba contra MySQL 10

7.2 Pruebas automáticas 11

7.3 Registro e inicio de sesión en la tienda 12

7.4 Caso de uso del cliente: compra con su cuenta 14

7.5 Documentación de la API y permisos 17

8. Limitaciones **21**

9. Referencias **22**

1. Introducción

Solsticio es la API REST del sistema multisede de inventario y ventas de una tienda de ropa con sedes en Bogotá. Hasta la Entrega 2 solo el personal (administradores y trabajadores) tenía cuenta, y la compra web se hacía sin registrarse. Este documento presenta la modificación de la aplicación para que **los usuarios se registren e inicien sesión contra la base de datos relacional MySQL**, con contraseñas protegidas mediante hashing, y para que el acceso a la API quede controlado por tokens JWT y por roles.

Además del código, el documento incluye las instrucciones para ejecutar el proyecto desde cero y las evidencias de la ejecución: la interfaz web, el script de prueba que lee directamente las filas guardadas en MySQL, las pruebas automáticas y la documentación interactiva de la API.

2. Objetivos

- Registrar clientes en MySQL (tablas *usuario* y *cliente*) sin guardar nunca la contraseña en claro.
- Autenticar a los usuarios comparando la contraseña con su hash bcrypt almacenado en la base.
- Proteger las rutas de la API con tokens JWT firmados y con expiración.
- Definir tres niveles de acceso (ADMINISTRADOR, TRABAJADOR y CLIENTE) y restringir cada caso de uso según el rol.
- Dejar evidencia verificable: un script de prueba, pruebas automáticas y capturas de la ejecución.

3. Modelo relacional de la autenticación

La autenticación usa tres tablas del esquema *database/solsticio_schema.sql*. No se agregaron tablas ni columnas: el modelo ya separaba a la persona con acceso (*usuario*) del comprador (*cliente*), y la llave *cliente.id_usuario* (única y opcional) permite que un comprador tenga o no cuenta. El único cambio de datos es el nuevo registro del rol CLIENTE.

Tabla	Columnas relevantes	Papel en la autenticación
rol	id_rol (PK), nombre (UNIQUE), descripcion	Catálogo de roles: ADMINISTRADOR, TRABAJADOR y, desde esta entrega, CLIENTE.
usuario	id_usuario (PK), id_rol (FK), id_sede (FK, nula), documento y email (UNIQUE), hash_password, activo	Credenciales. hash_password guarda el resultado de bcrypt (60 caracteres). id_sede es nula para el administrador global y para el cliente.
cliente	id_cliente (PK), id_usuario (FK, UNIQUE, nula), numero_documento (UNIQUE), nombres, email, telefono	Ficha del comprador. Se enlaza al usuario cuando el cliente se registra; sigue siendo nula para quien compra sin cuenta.

Para una base creada antes de esta entrega se incluye la migración *database/migraciones/001_rol_cliente.sql*, que agrega el rol sin borrar datos y se puede ejecutar varias veces:

```
INSERT INTO rol (nombre, descripcion)
SELECT 'CLIENTE', 'Comprador registrado en la tienda web'
WHERE NOT EXISTS (SELECT 1 FROM rol WHERE nombre = 'CLIENTE');
```

```
SELECT id_rol, nombre, descripcion FROM rol ORDER BY id_rol;
```

4. Script modificado: qué cambió en el código

La modificación está en la rama *entrega3-seguridad* del repositorio. Estos son los archivos tocados:

Archivo	Cambio
api/servicios/usuarios.py	Nueva función registrar_cliente(): crea usuario (rol CLIENTE, hash bcrypt) y su ficha en cliente en una transacción. Validación común de correo y contraseña (8 a 72 caracteres, letras y números). El login compara contra un hash de relleno si el correo no existe.
api/seguridad.py	Rol CLIENTE y funciones es_personal() / es_cliente(). Los decoradores exponen sus roles para la documentación.
api/rutas/auth.py	Nuevo endpoint POST /api/v1/auth/registro, que devuelve el token igual que el login.
api/rutas/cuenta.py (nuevo)	GET /api/v1/cuenta/pedidos y /cuenta/pedidos/{id}: solo rol CLIENTE y solo sus pedidos.
api/servicios/pedidos.py, api/rutas/pedidos.py	La venta y el pago en tienda exigen rol del personal (antes bastaba cualquier token). El pedido web de un cliente con sesión se enlaza a su ficha.
api/documentacion.py (nuevo)	Especificación OpenAPI 3.0 y Swagger UI en /api/docs.
web/cuenta.html, cuenta.js, carrito.js, panel/app.js	Páginas de registro, inicio de sesión y Mis pedidos; carrito con la cuenta; el panel rechaza a los clientes.
database/…	Rol CLIENTE en el script de creación y migración 001.
scripts/prueba_autenticacion.py (nuevo)	Script de prueba de punta a punta (secciones 6.7 y 7.1).
tests/test_seguridad.py, test_documentacion.py (nuevos)	35 pruebas automáticas de autenticación, tokens, roles y documentación.

4.1 Registro contra MySQL

El registro valida los datos, crea la fila de *usuario* con el hash bcrypt y la ficha de *cliente*. Si el documento ya tenía compras sin cuenta, solo se enlaza cuando el correo coincide, para que nadie reclame el historial de otra persona conociendo solo su documento:

```
def registrar_cliente(datos):
    """Registro público de la tienda web. Crea el usuario con rol CLIENTE y su ficha
    en cliente, enlazadas por cliente.id_usuario, en una sola transacción.

    Si el comprador ya había comprado sin cuenta (existe en cliente con ese documento),
    la cuenta nueva se enlaza a esa ficha para que vea sus compras anteriores, pero
    solo si el email coincide con el que dejó al comprar; así nadie reclama el
    historial de otra persona conociendo solo su documento.
    """
    exigir(datos, "documento", "nombres", "apellidos", "email", "password")
    email = _validar_email(datos)
    password = _validar_password(datos)
    documento = texto(datos, "documento", 20, obligatorio=True)
    _exigir_unicos(documento, email)
    rol = db.session.scalars(select(Rol).where(Rol.nombre == CLIENTE)).first()
    if rol is None:
```

```
        raise ErrorApi(500, "Falta el rol CLIENTE en la base. Ejecute database/migraciones/001_rol_cliente.sql")

    nombres = texto(datos, "nombres", 80, obligatorio=True)
    apellidos = texto(datos, "apellidos", 80, obligatorio=True)
    usuario = Usuario(
        id_rol=rol.id_rol, id_sede=None, documento=documento, nombres=nombres,
        apellidos=apellidos, email=email, hash_password=hashear_password(password), activo=True,
    )
    db.session.add(usuario)
    db.session.flush()

    cliente = db.session.scalars(select(Cliente).where(Cliente.numero_documento == documento)).first()
    if cliente is None:
        cliente = Cliente(
            tipo_documento=a_enum(TipoDocumento, datos.get("tipo_documento", "CC"), "tipo_documento"),
            numero_documento=documento, nombres=nombres, apellidos=apellidos, email=email,
        )
        db.session.add(cliente)
    elif cliente.id_usuario is not None or (cliente.email or "").lower() != email:
        db.session.rollback()
        raise ErrorApi(409, "Ese documento ya tiene compras con otro correo. "
                            "Acérquese a una sede para vincular su cuenta.")
    cliente.id_usuario = usuario.id_usuario
    for campo, maximo in (("telefono", 20), ("direccion", 150)):
        valor = texto(datos, campo, maximo)
        if valor:
            setattr(cliente, campo, valor)
    db.session.commit()
    return usuario
```

4.2 Inicio de sesión y hashing

bcrypt genera una sal aleatoria por contraseña y la guarda dentro del propio hash. Al iniciar sesión se busca el correo en MySQL y se compara con *bcrypt.checkpw*. La respuesta es la misma si el correo no existe o si la contraseña es incorrecta:

```
def hashear_password(password):
    return bcrypt.hashpw(password.encode("utf-8"), bcrypt.gensalt()).decode("utf-8")

def verificar_password(password, hash_password):
    if hash_password is None:
        hash_password = _HASH_RELLENO
        bcrypt.checkpw(password.encode("utf-8"), hash_password.encode("utf-8"))
        return False
    try:
        return bcrypt.checkpw(password.encode("utf-8"), hash_password.encode("utf-8"))
    except ValueError:
        return False

def autenticar(datos):
    exigir(datos, "email", "password")
    usuario = db.session.scalars(
        select(Usuario).where(Usuario.email == str(datos["email"]).strip().lower())
    ).first()
    valida = verificar_password(str(datos["password"]), usuario.hash_password if usuario else None)
    if usuario is None or not usuario.activo or not valida:
        raise ErrorApi(401, "Credenciales incorrectas")
    return usuario
```

4.3 Token JWT y protección de rutas

El token es un JWT HS256 firmado con SECRET_KEY, con el id del usuario, su rol, su sede, la fecha de emisión y la de expiración. El decorador *requiere_auth* verifica la firma y la expiración. Luego **vuelve a leer al usuario de MySQL** en cada petición, así que una desactivación o un cambio de rol tienen efecto inmediato:

```
def crear_token(usuario):
    minutos = current_app.config["ACCESS_TOKEN_EXPIRE_MINUTES"]
    ahora = datetime.now(timezone.utc)
    payload = {
        "sub": str(usuario.id_usuario),
        "rol": usuario.rol.nombre,
        "id_sede": usuario.id_sede,
        "iat": ahora,
        "exp": ahora + timedelta(minutes=minutos),
    }
    token = jwt.encode(payload, current_app.config["SECRET_KEY"], algorithm="HS256")
    return token, minutos * 60

def requiere_auth(*roles):
    """Exige token válido. Si se indican roles, el usuario debe tener uno de ellos."""
    def decorador(vista):
        @wraps(vista)
        def envoltura(*args, **kwargs):
            g.usuario = _usuario_desde_token(obligatorio=True)
            if roles and g.usuario.rol.nombre not in roles:
                raise ErrorApi(403, "No tiene permisos para esta operación")
            return vista(*args, **kwargs)
        # La documentación OpenAPI (api/documentacion.py) lee estos atributos para
        # mostrar quién puede usar cada ruta sin repetir la información a mano.
        envoltura.roles_permitidos = roles or ROLES
        return envoltura
    return decorador
```

5. Roles y permisos (RBAC)

Cada ruta declara los roles que admite, por ejemplo *@requiere_auth(ADMINISTRADOR)* o *@requiere_auth(CLIENTE)*. Sin token la API responde 401. Con un rol no permitido responde 403. La matriz por caso de uso queda así:

Caso de uso	Público	CLIENTE	TRABAJADOR	ADMIN.
Explorar catálogo, escanear QR, disponibilidad	Sí	Sí	Sí	Sí
Compra virtual (pedido WEB y pago)	Con sus datos	Con su cuenta	-	-
Ver mis pedidos y su estado	-	Sí	-	-
Venta en tienda, cobro y factura	-	-	Su sede	Sí
Stock, ingresos, despachar y recibir traslados	-	-	Su sede	Sí
Ajustes, movimientos y crear traslados	-	-	-	Sí
Productos, categorías, sedes y usuarios	-	-	-	Sí
Moderar reseñas, bitácora y reportes	-	-	-	Sí

Situación	Respuesta
Sin token, firma alterada, firmado con otra clave, alg none o vencido	401
Token válido con un rol sin permiso	403
Trabajador que pide datos de otra sede	403
Cliente que pide un pedido ajeno	404 (no se confirma que exista)

6. Instrucciones para ejecutar

Requisitos: Python 3.12 o superior, MySQL Server 8.0 y Git. MongoDB 7 o superior es necesario para las partes NoSQL de la Entrega 2 (reseñas, fichas técnicas y bitácora), pero **no para la autenticación**: sin MongoDB la API arranca y el registro, el inicio de sesión y los roles funcionan igual.

6.1 Descargar el proyecto

```
git clone https://github.com/miguelamarinb-ops/Solsticio.git
cd Solsticio
git checkout entrega3-seguridad
```

6.2 Crear la base de datos

Base nueva (borra la base *solsticio* si existía), desde cmd o Git Bash:

```
mysql -u root -p < database/solsticio_schema.sql
```

Base existente de una entrega anterior (no borra datos):

```
mysql -u root -p solsticio < database/migraciones/001_rol_cliente.sql
```

En MySQL Workbench también sirve: *File → Open SQL Script*, elegir el archivo y ejecutarlo con el rayo.

6.3 Entorno virtual y dependencias

```
python -m venv .venv
.venv\Scripts\activate          # Windows (PowerShell o cmd)
pip install -r requirements.txt -r requirements-dev.txt
```

6.4 Variables de entorno

Copiar *.env.example* a *.env* y completar la conexión y la clave del token:

```
DATABASE_URL=mysql+pymysql://root:TU_CLAVE@localhost:3306/solsticio
SECRET_KEY=una-clave-larga-y-aleatoria
ACCESS_TOKEN_EXPIRE_MINUTES=60
MONGO_URI=mongodb://localhost:27017
```

Una SECRET_KEY segura se genera con *python -c "import secrets; print(secrets.token_hex(32))"*.

6.5 Primer administrador y datos de ejemplo

```
flask --app app verificar-bd
flask --app app crear-admin
flask --app app sembrar-demo            # o --sin-mongo si no hay MongoDB
```

6.6 Levantar el servidor

```
python app.py
```

- Tienda y registro de clientes: http://127.0.0.1:5000/cuenta
- Panel del personal: http://127.0.0.1:5000/panel
- Documentación de la API (Swagger UI): http://127.0.0.1:5000/api/docs

6.7 Ejecutar el script de prueba y las pruebas automáticas

Con el servidor encendido, en otra terminal (usa las credenciales del administrador creado en 6.5):

```
$env:SOLSTICIO_EMAIL="admin@solsticio.com"; $env:SOLSTICIO_PASSWORD="tu-clave"
python scripts/prueba_autenticacion.py
```

Las pruebas automáticas usan una base aparte (*solsticio_test*) y no tocan los datos de desarrollo:

```
pytest -m solo_mysql        # 35 pruebas: seguridad y documentación (solo MySQL)
pytest                      # las 82 pruebas, con MySQL y MongoDB encendidos
```

7. Evidencias de la ejecución

Las capturas se tomaron el 4 de noviembre de 2026 con la API en local sobre una base aparte (*solsticio_e2e*, catálogo de ejemplo). También están en *docs/imagenes/entrega3/* del repositorio.

7.1 Script de prueba contra MySQL

El script registra un cliente y lee **directamente de MySQL** la fila guardada, donde se ve el hash bcrypt ($2b$12$…) en lugar de la contraseña. Después inicia sesión, decodifica el JWT y prueba los rechazos 401 y la matriz de roles. Las 23 comprobaciones terminan en OK.

```
PS D:\UNIVERSIDAD\ProyectoNTD\Solsticio> python scripts\prueba_autenticacion.py

Solsticio: autenticación contra MySQL, JWT y roles  http://127.0.0.1:5001/api/v1

1. Registro de cliente
  OK  Registro con datos válidos                POST  /auth/registro              -> 201 (esperado 201)  rol=CLIENTE
  OK  Mismo documento y correo otra vez         POST  /auth/registro              -> 409 (esperado 409)  Ya existe un usuario con ese documento o email
  OK  Contraseña sin números                    POST  /auth/registro              -> 422 (esperado 422)  La contraseña debe combinar letras y números

2. Qué quedó guardado en MySQL
  SELECT ... FROM usuario JOIN rol LEFT JOIN cliente WHERE email = 'laura.3851216@correo.com'
    id_usuario       3
    rol              CLIENTE
    id_sede          None
    email            laura.3851216@correo.com
    hash_password    $2b$12$TeaCizYbfvBJzRbHvKfituA9HFp3HUOuiT/pJWBT.iWP6Cdz8ZS8a
    id_cliente       2
    numero_documento  103851216
  El hash empieza por $2b$ (bcrypt con sal y costo 12): la contraseña nunca se guarda en claro.

3. Inicio de sesión
  OK  Correo y contraseña correctos             POST  /auth/login                 -> 200 (esperado 200)
  OK  Contraseña equivocada                     POST  /auth/login                 -> 401 (esperado 401)  Credenciales incorrectas
  OK  Correo que no existe (mismo mensaje)      POST  /auth/login                 -> 401 (esperado 401)  Credenciales incorrectas

4. Contenido del token JWT del cliente
    cabecera  {"alg": "HS256", "typ": "JWT"}
    payload   {"sub": "3", "rol": "CLIENTE", "id_sede": null, "iat": 1793851217, "exp": 1793854817}
    firma     aStx3QwIfzSeVqxeZAk2...  (HMAC-SHA256 con SECRET_KEY: sin la clave no se puede falsificar)

5. Rutas protegidas
  OK  Perfil sin token                          GET   /auth/perfil                -> 401 (esperado 401)
  OK  Perfil con la firma alterada              GET   /auth/perfil                -> 401 (esperado 401)
  OK  Token editado para decir ADMINISTRADOR    GET   /usuarios                   -> 401 (esperado 401)
  OK  Perfil con el token correcto              GET   /auth/perfil                -> 200 (esperado 200)

6. Roles y permisos (RBAC)
  OK  CLIENTE ve sus pedidos                    GET   /cuenta/pedidos             -> 200 (esperado 200)
  OK  CLIENTE consulta stock                    GET   /stock                      -> 403 (esperado 403)
  OK  CLIENTE lista usuarios                    GET   /usuarios                   -> 403 (esperado 403)
  OK  TRABAJADOR consulta stock de su sede      GET   /stock                      -> 200 (esperado 200)
  OK  TRABAJADOR consulta stock de otra sede    GET   /stock?id_sede=2            -> 403 (esperado 403)
  OK  TRABAJADOR lista usuarios                 GET   /usuarios                   -> 403 (esperado 403)
  OK  TRABAJADOR entra a 'mis pedidos'          GET   /cuenta/pedidos             -> 403 (esperado 403)
  OK  ADMINISTRADOR lista usuarios              GET   /usuarios                   -> 200 (esperado 200)
  OK  ADMINISTRADOR consulta stock de otra sede GET   /stock?id_sede=2            -> 200 (esperado 200)

Todo correcto
```

Figura 1. Salida de scripts/prueba_autenticacion.py: registro, fila en MySQL con el hash, JWT, 401 y 403.

7.2 Pruebas automáticas

```
PS D:\UNIVERSIDAD\ProyectoNTD\Solsticio> pytest -m solo_mysql -v

============================= test session starts =============================
platform win32 -- Python 3.14.7, pytest-9.2.0, pluggy-1.7.0 -- D:\UNIVERSIDAD\ProyectoNTD\Solsticio\.venv\Scripts\python.exe
rootdir: D:\UNIVERSIDAD\ProyectoNTD\Solsticio
configfile: pytest.ini
testpaths: tests
collecting ... collected 82 items / 47 deselected / 35 selected

tests/test_documentacion.py::test_todas_las_rutas_estan_documentadas PASSED [  2%]
tests/test_documentacion.py::test_especificacion_openapi_y_permisos PASSED [  5%]
tests/test_documentacion.py::test_swagger_ui PASSED                      [  8%]
tests/test_seguridad.py::test_registro_crea_usuario_cliente_con_hash_bcrypt PASSED [ 11%]
tests/test_seguridad.py::test_registro_rechaza_duplicados_y_datos_invalidos PASSED [ 14%]
tests/test_seguridad.py::test_login_correcto_e_incorrecto PASSED         [ 17%]
tests/test_seguridad.py::test_registro_enlaza_compras_previas_solo_con_el_mismo_correo PASSED [ 20%]
tests/test_seguridad.py::test_rutas_protegidas_exigen_token_valido PASSED [ 22%]
tests/test_seguridad.py::test_el_rol_se_lee_de_la_base_y_no_del_token PASSED [ 25%]
tests/test_seguridad.py::test_usuario_desactivado_pierde_el_acceso PASSED [ 28%]
tests/test_seguridad.py::test_cliente_no_entra_a_rutas_del_personal[get-/usuarios] PASSED [ 31%]
tests/test_seguridad.py::test_cliente_no_entra_a_rutas_del_personal[post-/usuarios] PASSED [ 34%]
tests/test_seguridad.py::test_cliente_no_entra_a_rutas_del_personal[post-/productos] PASSED [ 37%]
tests/test_seguridad.py::test_cliente_no_entra_a_rutas_del_personal[post-/categorias] PASSED [ 40%]
tests/test_seguridad.py::test_cliente_no_entra_a_rutas_del_personal[post-/sedes] PASSED [ 42%]
tests/test_seguridad.py::test_cliente_no_entra_a_rutas_del_personal[post-/inventario/ajustes] PASSED [ 45%]
tests/test_seguridad.py::test_cliente_no_entra_a_rutas_del_personal[get-/inventario/movimientos] PASSED [ 48%]
tests/test_seguridad.py::test_cliente_no_entra_a_rutas_del_personal[post-/traslados] PASSED [ 51%]
tests/test_seguridad.py::test_cliente_no_entra_a_rutas_del_personal[get-/stock] PASSED [ 54%]
tests/test_seguridad.py::test_cliente_no_entra_a_rutas_del_personal[get-/pedidos] PASSED [ 57%]
tests/test_seguridad.py::test_cliente_no_entra_a_rutas_del_personal[get-/traslados] PASSED [ 60%]
tests/test_seguridad.py::test_cliente_no_entra_a_rutas_del_personal[post-/inventario/ingresos] PASSED [ 62%]
tests/test_seguridad.py::test_trabajador_no_entra_a_rutas_del_administrador[get-/usuarios] PASSED [ 65%]
tests/test_seguridad.py::test_trabajador_no_entra_a_rutas_del_administrador[post-/usuarios] PASSED [ 68%]
tests/test_seguridad.py::test_trabajador_no_entra_a_rutas_del_administrador[post-/productos] PASSED [ 71%]
tests/test_seguridad.py::test_trabajador_no_entra_a_rutas_del_administrador[post-/categorias] PASSED [ 74%]
tests/test_seguridad.py::test_trabajador_no_entra_a_rutas_del_administrador[post-/sedes] PASSED [ 77%]
tests/test_seguridad.py::test_trabajador_no_entra_a_rutas_del_administrador[post-/inventario/ajustes] PASSED [ 80%]
tests/test_seguridad.py::test_trabajador_no_entra_a_rutas_del_administrador[get-/inventario/movimientos] PASSED [ 82%]
tests/test_seguridad.py::test_trabajador_no_entra_a_rutas_del_administrador[post-/traslados] PASSED [ 85%]
tests/test_seguridad.py::test_trabajador_opera_solo_en_su_sede PASSED    [ 88%]
tests/test_seguridad.py::test_administrador_entra_a_todo PASSED          [ 91%]
tests/test_seguridad.py::test_admin_no_puede_crear_clientes_por_la_ruta_del_personal PASSED [ 94%]
tests/test_seguridad.py::test_cliente_no_registra_ventas_ni_pagos_de_tienda PASSED [ 97%]
tests/test_seguridad.py::test_cliente_compra_con_su_cuenta_y_ve_solo_sus_pedidos PASSED [100%]

================ 35 passed, 47 deselected in 125.18s (0:02:05) ================
```

Figura 2. pytest -m solo_mysql: 35 pruebas aprobadas.

7.3 Registro e inicio de sesión en la tienda

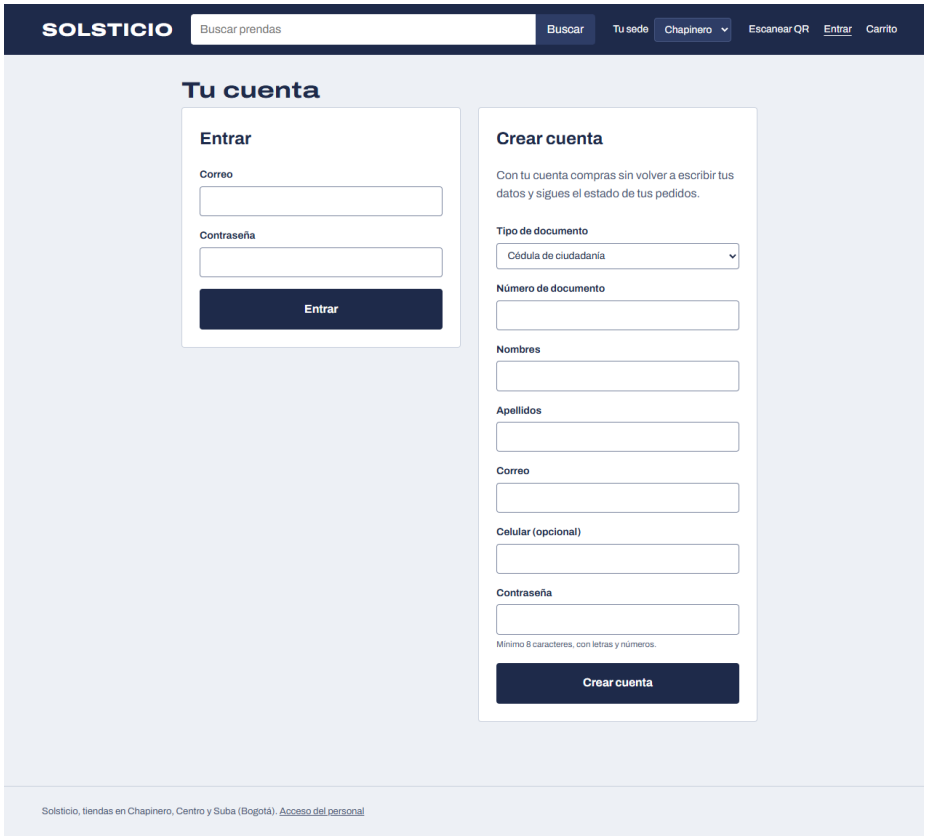


Figura 3. Página /cuenta: entrar o crear cuenta.

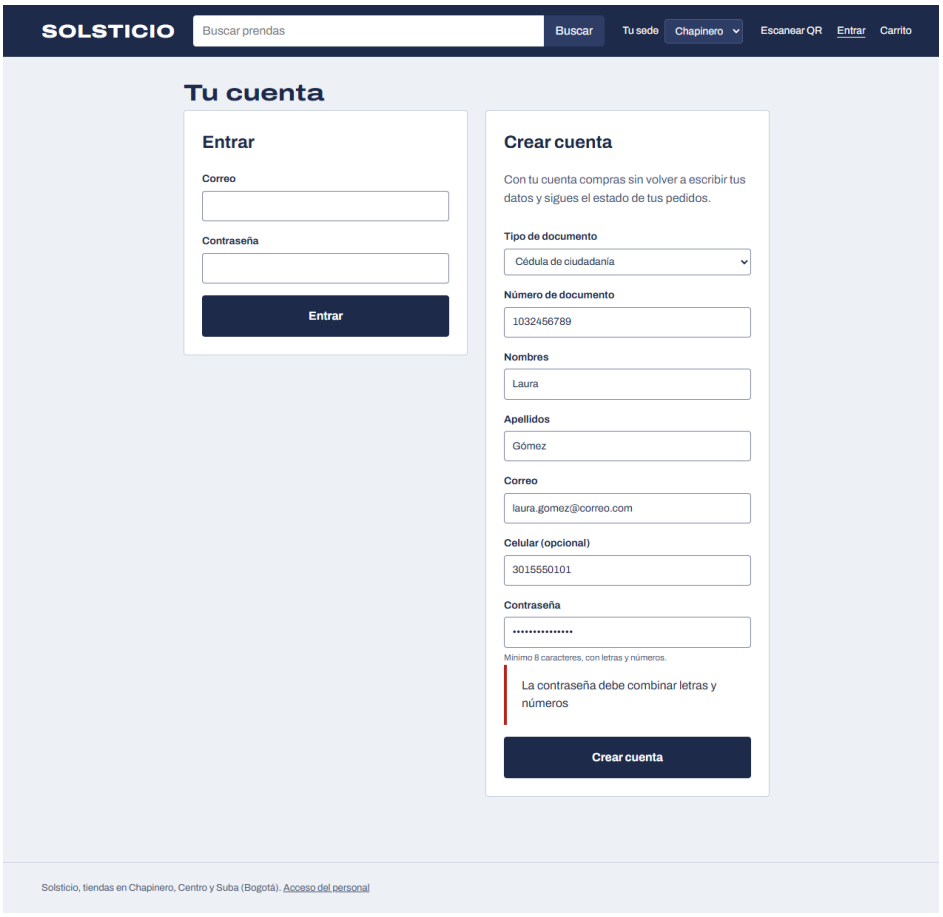


Figura 4. El servidor rechaza una contraseña sin números (422).

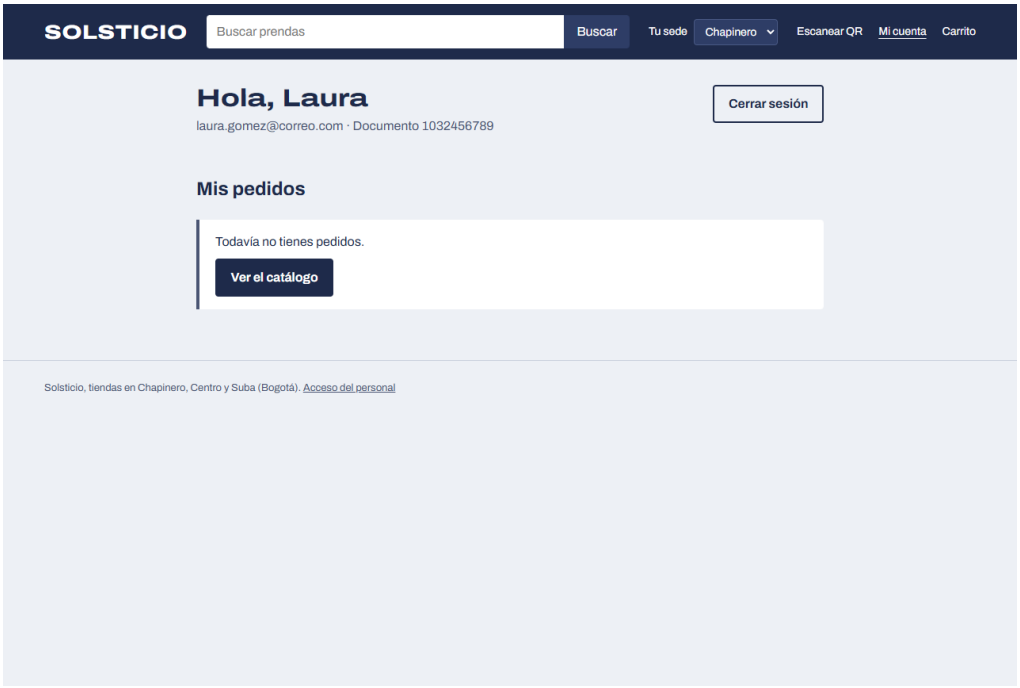


Figura 5. Cuenta creada: la sesión queda iniciada con el token devuelto por el registro.

7.4 Caso de uso del cliente: compra con su cuenta

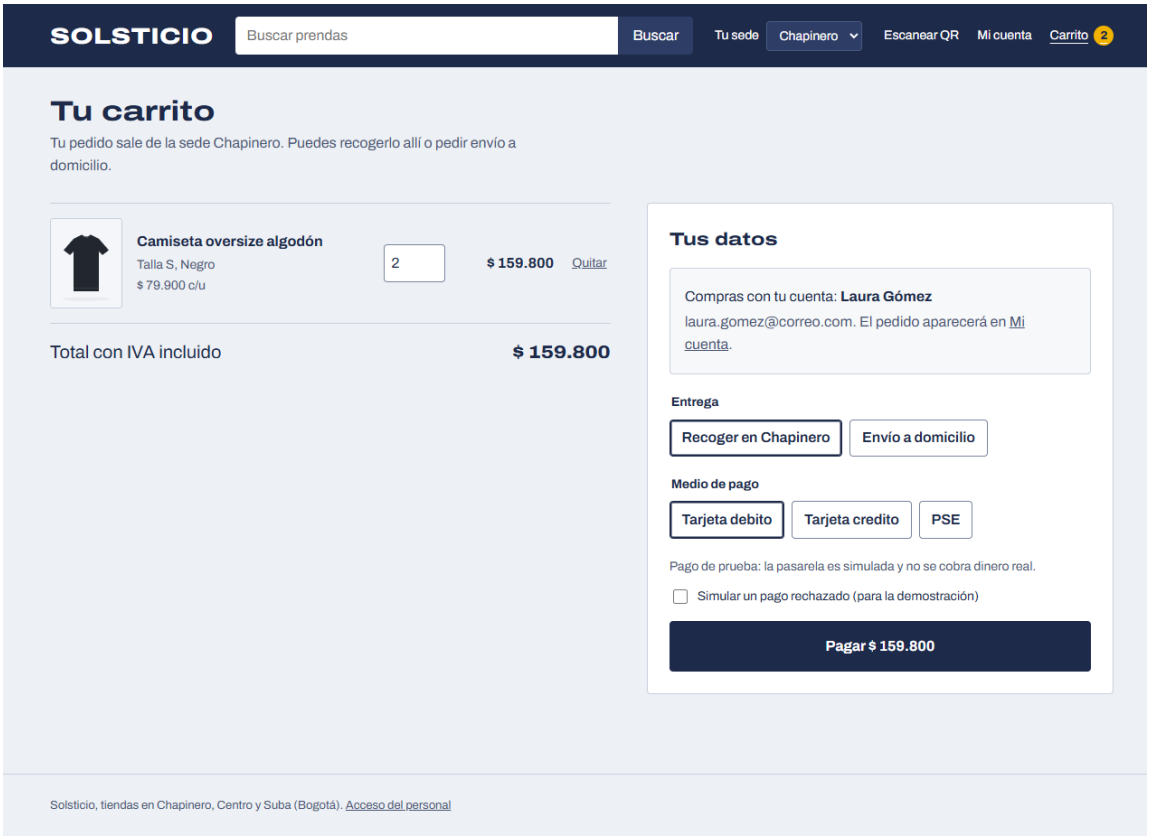


Figura 6. Con la sesión iniciada, el carrito no vuelve a pedir los datos del comprador.

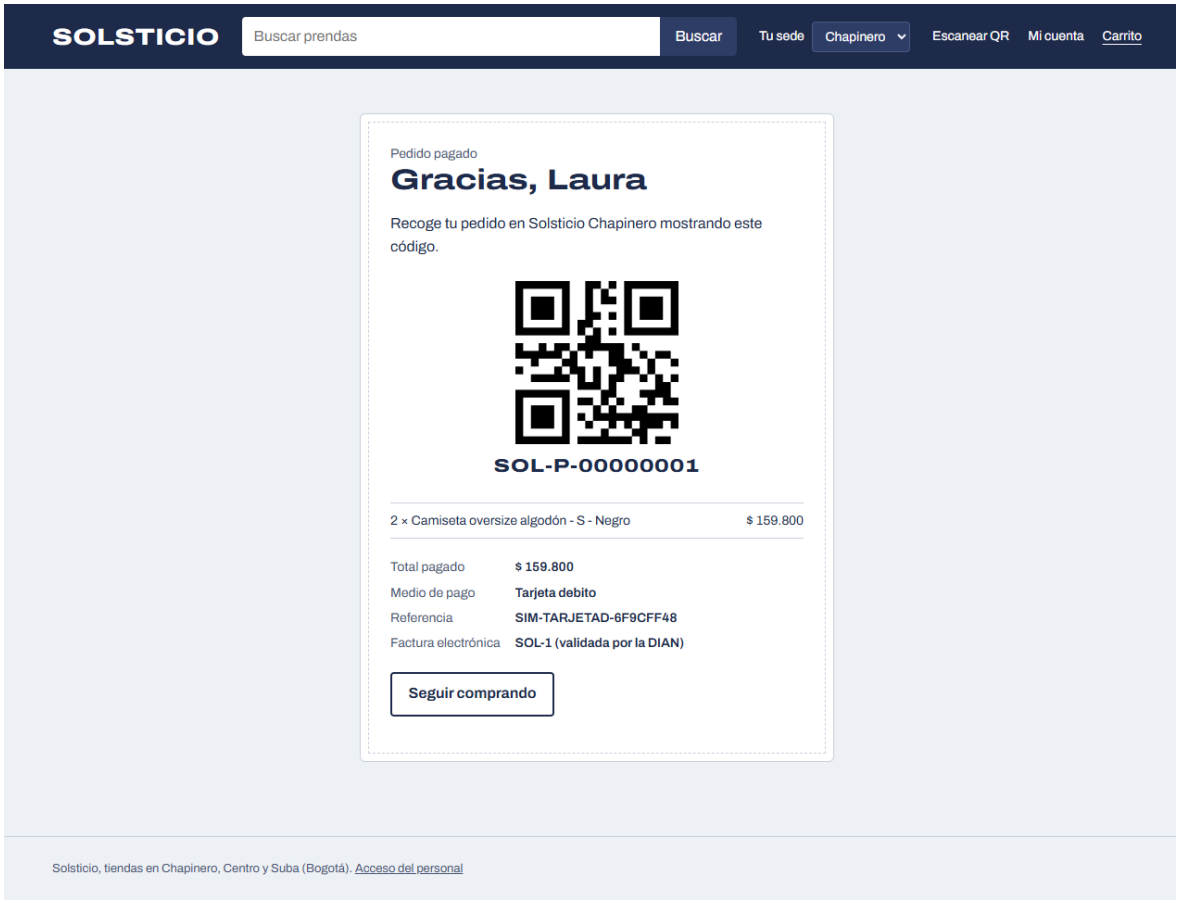


Figura 7. Pedido pagado con la pasarela simulada.

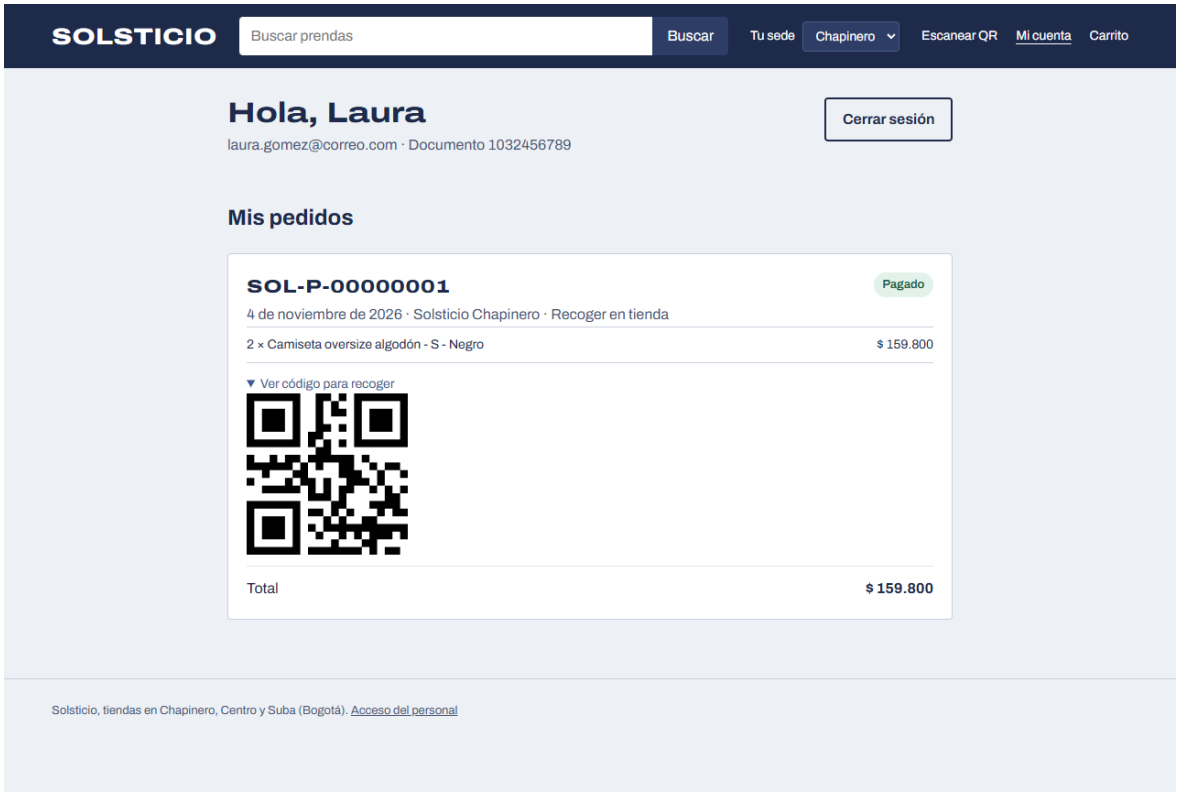


Figura 8. Mis pedidos: el cliente solo ve los suyos, con estado y código QR para recoger.

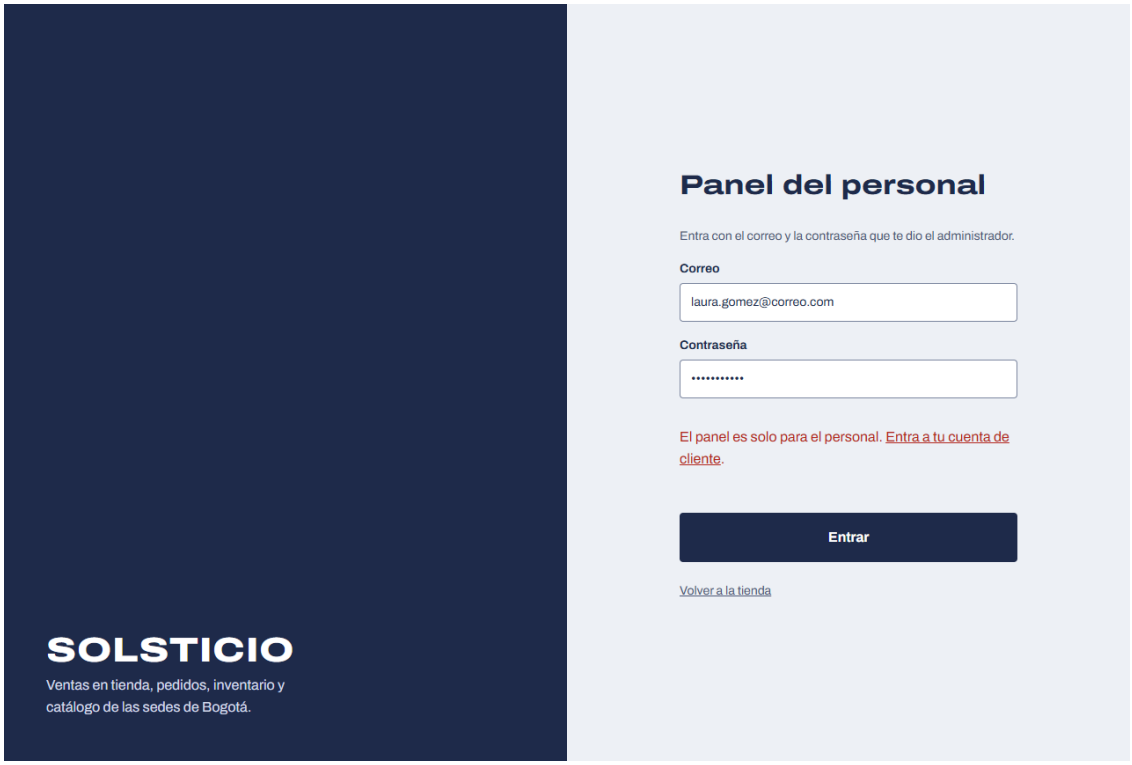


Figura 9. Un cliente que intenta entrar al panel del personal es rechazado.

7.5 Documentación de la API y permisos

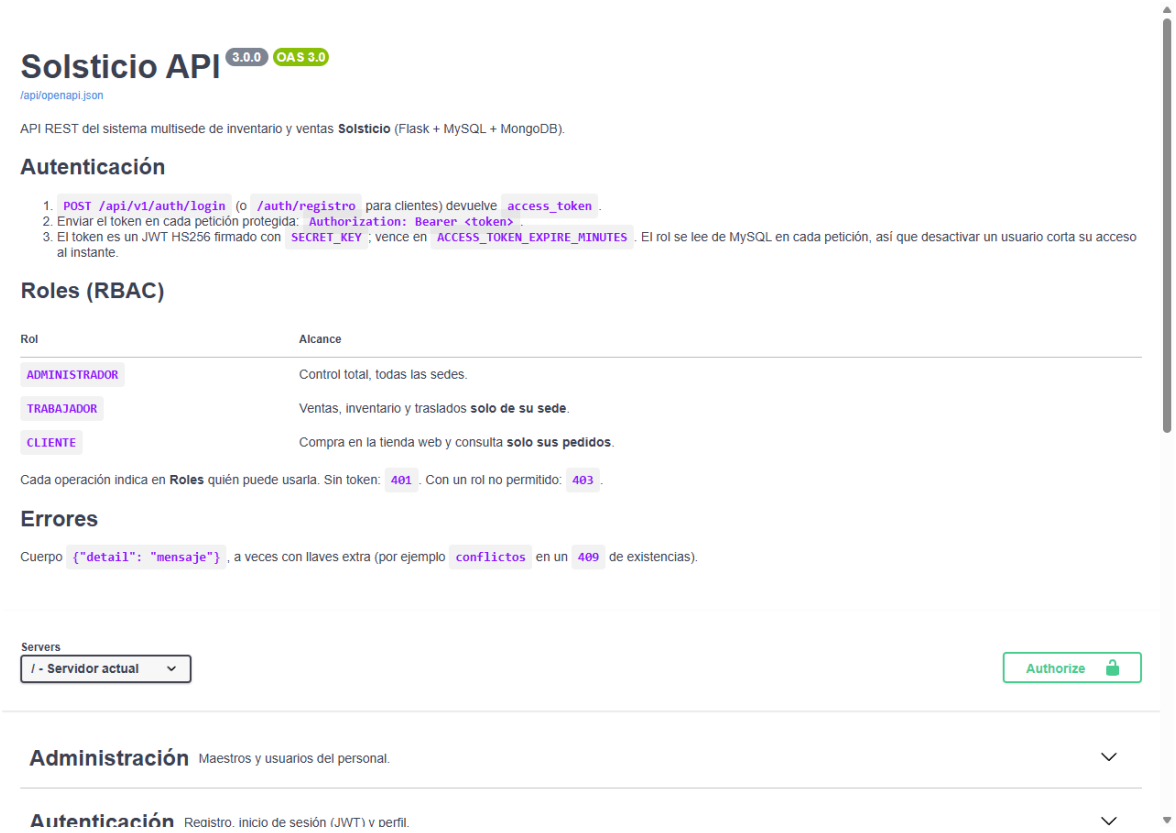


Figura 10. Swagger UI en /api/docs: autenticación, roles y formato de errores.

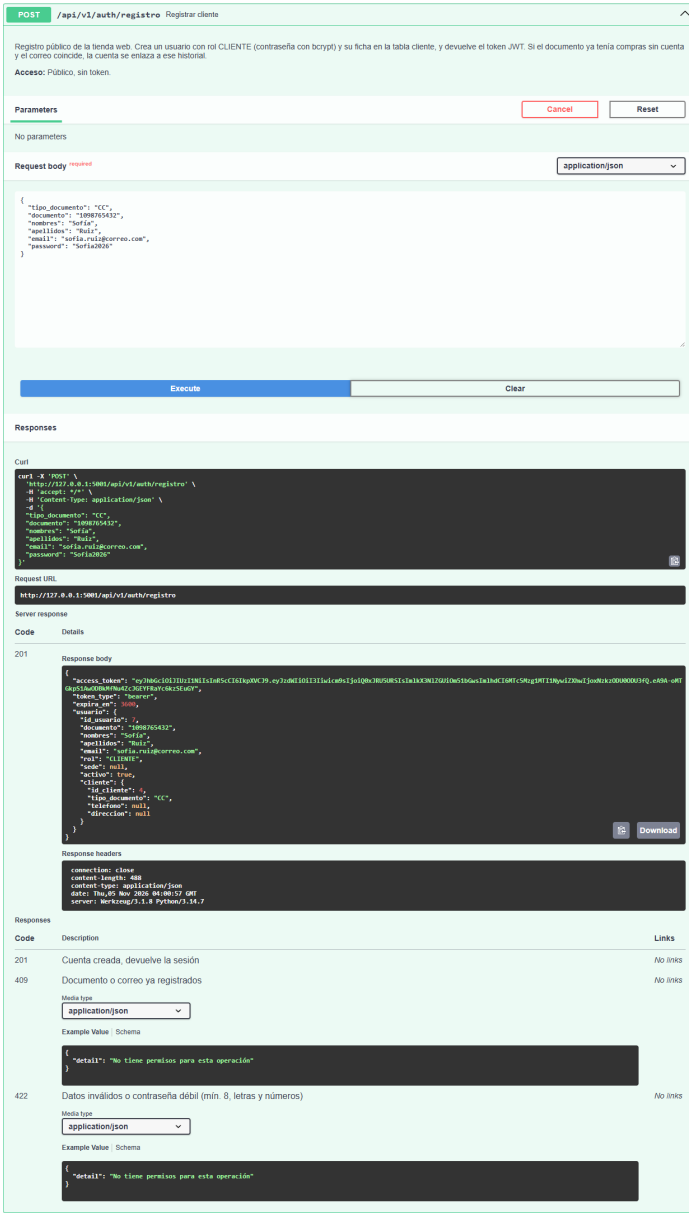


Figura 11. POST /auth/registro desde Swagger: 201 con el access_token.

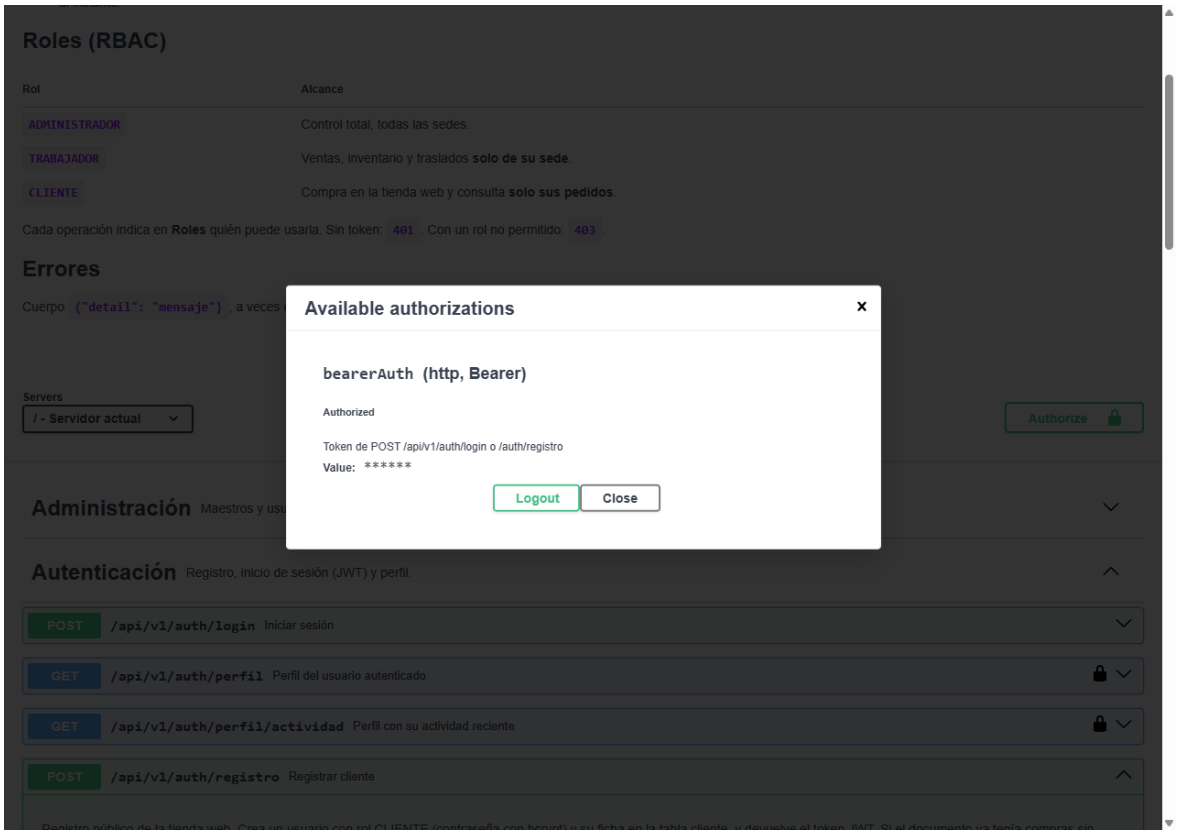


Figura 12. Autorización con el token del cliente (Bearer).

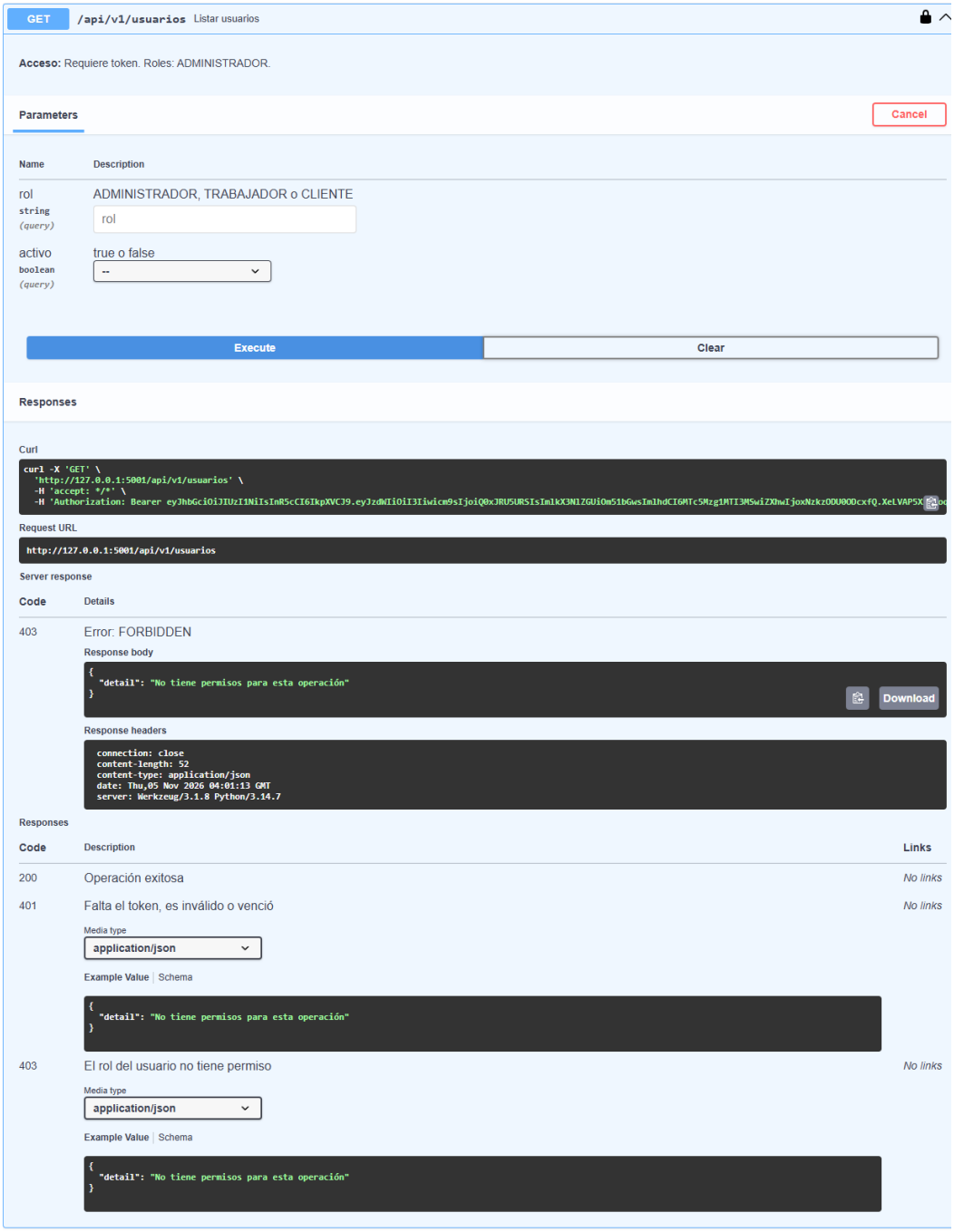


Figura 13. El cliente pide GET /usuarios (solo ADMINISTRADOR): 403.

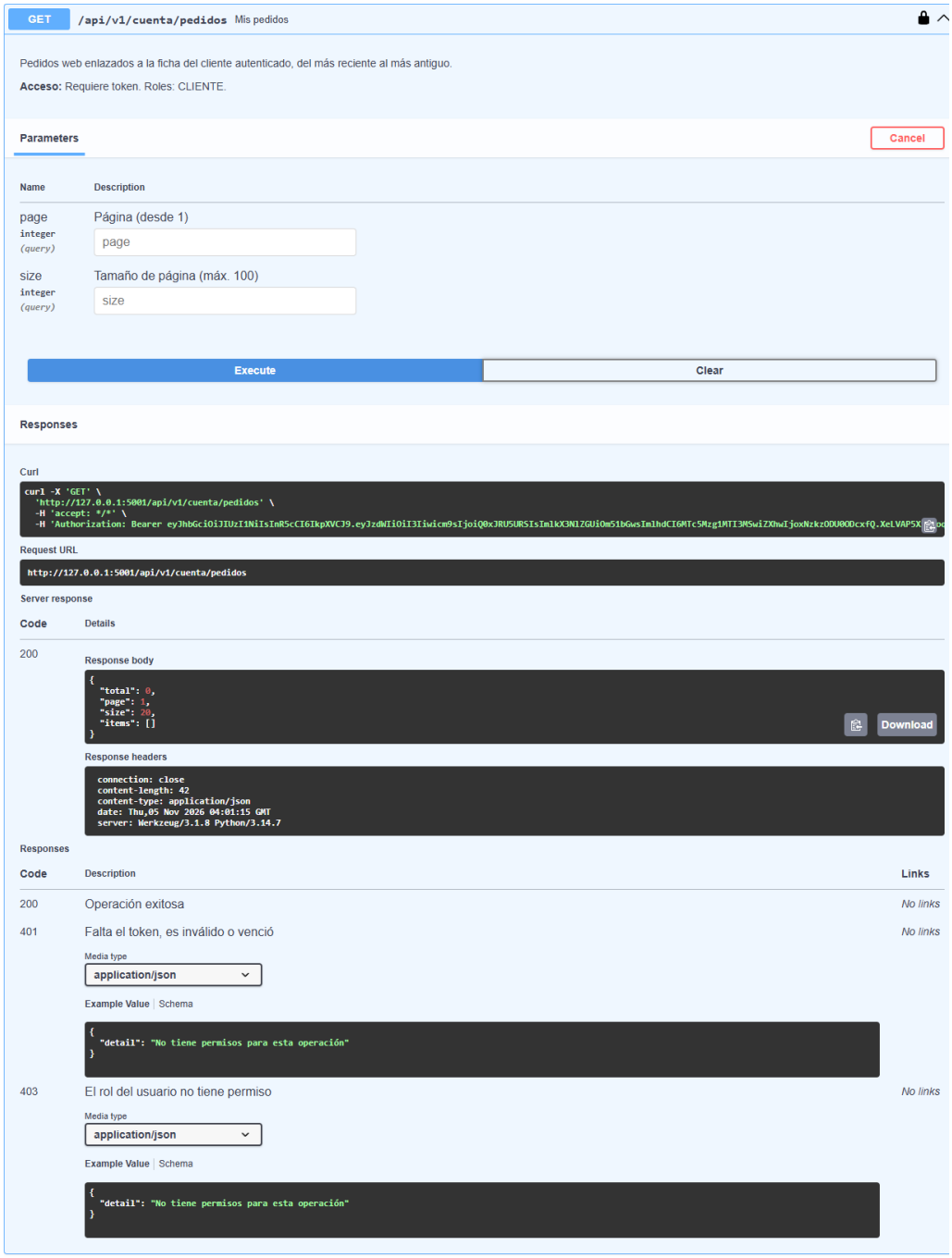


Figura 14. El cliente pide GET /cuenta/pedidos: 200 con sus pedidos.

8. Limitaciones

- El token no se puede revocar antes de que venza. Desactivar al usuario sí corta el acceso de inmediato, porque el usuario se relee de MySQL en cada petición.
- No hay token de renovación (refresh token). Al vencer, el usuario vuelve a iniciar sesión.
- No hay límite de intentos de inicio de sesión. En producción convendría limitar por IP y por cuenta.

- La tienda guarda el token del cliente en localStorage. El contenido que escriben los usuarios se inserta siempre como texto (textContent), lo que reduce el riesgo de XSS. Una cookie HttpOnly sería más robusta.
- Las capturas se tomaron sin MongoDB. Las pruebas que dependen de MongoDB (47) deben correrse en un equipo con MongoDB encendido.

9. Referencias

- Jones, M., Bradley, J. y Sakimura, N. (2015). *RFC 7519: JSON Web Token (JWT)*. IETF.
- Provos, N. y Mazières, D. (1999). A Future-Adaptable Password Scheme. *USENIX Annual Technical Conference*.
- OWASP Foundation. *Password Storage Cheat Sheet* y *Authentication Cheat Sheet*.
- OWASP Foundation. *Authorization Cheat Sheet* (control de acceso basado en roles).
- Pallets Projects. *Flask Documentation* (3.0). PyJWT y bcrypt para Python.
- OpenAPI Initiative. *OpenAPI Specification 3.0.3*.